ApplyPilot: A Private Agentic Assistant for Application Search, CV Tailoring, Evaluation, Email Sending, and Reply Tracking

Muhammad Usman and Muhammad Umar
Department of Electrical Engineering and Computer Science
Gwangju Institute of Science and Technology, Korea

Abstract:

Applying to academic and professional opportunities requires repeated manual work, including searching through noisy web pages, tailoring curriculum vitae (CVs), drafting cover letters, sending emails, and tracking replies. This process is time-consuming and error-prone, especially when applicants must prepare multiple customized applications. This paper presents ApplyPilot, a private agentic assistant that automates and organizes the application workflow through a Telegram-based interface. ApplyPilot integrates real-time opportunity search, academic intent detection, CV tailoring, cover letter and email drafting, an Evaluation Agent, email sending, reply tracking, and auditable records. The system uses a local model through Ollama for private text generation tasks, Tavily for live search, Gmail SMTP/IMAP for communication, and CSV/usage logs for evidence tracking. An OpenClaw and MCP inspection layer is also implemented to expose safe project artifacts and status information without replacing the main Telegram action workflow. Functional evaluation showed that ApplyPilot can complete an end-to-end application workflow: upload a CV, search and rank opportunities, generate a tailored CV, revise and finalize the CV, draft application materials, evaluate the package, send an email with attachments, detect replies, and update tracker records. The results demonstrate that ApplyPilot reduces repetitive application effort while preserving human approval for external actions.

I. Introduction

Students and early-career researchers often apply to multiple academic and professional opportunities, including Master's programs, PhD positions, internships, research assistant roles, and job openings. Each application usually requires searching through scattered job portals, university pages, laboratory websites, and email announcements. After identifying a relevant opportunity, the applicant must tailor the CV, prepare a cover letter, write an email, attach documents, send the application, and track replies. Although each step is simple in isolation, the complete workflow becomes repetitive and difficult to manage when repeated across many opportunities.

A major challenge is that application search results are often noisy and entirely depend on the search engine optimization. Web search engines may return results from unrelated countries, old listings, generic career advice pages, or pages that only partially match the intended field. Academic opportunities are even harder to identify because many positions are posted on university department pages, laboratory websites, professor pages, or informal "join us" sections rather than standard job boards. In addition, manually tailoring the CV and cover letter for every opportunity can lead to inconsistent quality. Applicants may also forget which applications were sent, whether a reply was received, and whether the reply was already reviewed.

To address these problems, we developed **ApplyPilot**, a private agentic application assistant. ApplyPilot is designed as a Telegram-based workflow assistant that supports both normal conversation and structured

application commands. The system automates key stages of the application pipeline: searching for opportunities, detecting academic intent, ranking results, preparing tailored CV drafts, revising and finalizing CVs, generating cover letters and emails, evaluating the final application package, sending emails after explicit approval, tracking replies, and recording all actions in logs and CSV trackers.

The main contribution of this project is not only the use of a language model, but the construction of a complete agentic workflow with state, tools, logs, approval gates, and evidence records. ApplyPilot combines local LLM-based generation with external search and email tools while maintaining user control over high-impact actions. In particular, the system includes an Evaluation Agent that checks the CV, cover letter, email body, and subject before sending. This provides a quality gate that reduces the risk of sending weak, incomplete, or unverified material.

II. System Design and Methodology

ApplyPilot was designed around a practical application workflow. The system starts from a user's CV and search goal, then produces ranked opportunities and application materials. The workflow is controlled through Telegram commands, allowing the user to interact with the assistant from a familiar messaging interface.

The main workflow is:

1. The user starts /campaign.
2. The user uploads a CV in PDF, DOCX, TXT format, or pastes CV text.
3. The user enters a search goal, such as "Computer vision PhD Germany."
4. The system detects whether the query is academic.
5. The search and ranking agents retrieve and prepare opportunities.
6. CV drafts are generated for the top opportunities.
7. The user views a CV using /showcampaigncv.
8. The user revises or finalizes the CV.
9. The system drafts the cover letter and email.
10. The Evaluation Agent checks the application package.
11. The user explicitly sends the email using /sendemail.
12. The reply tracker checks the inbox and updates the CSV tracker.

This methodology separates generation, evaluation, and action. The system may search, summarize, draft, and evaluate automatically, but it cannot send email unless the user explicitly gives the /sendemail command. This human approval gate is essential because email sending is an external action that may affect real recipients.

ApplyPilot also includes academic intent detection. If the user's query contains terms such as "PhD," "PHD," "MS," "Master," "doctoral," "scholarship," "professor," or "lab," the system switches to academic search mode. In this mode, the search query is expanded to prioritize universities, laboratories, professors, official academic vacancy pages, and funded doctoral or master's opportunities. This improves the suitability of results for academic applications.

III. Architecture and Implementation

The implementation follows a layered architecture, shown conceptually in Fig. 1. The first layer is the Telegram interface implemented mainly in bot.py, which is further renamed as ApplyPilot.py. It receives commands, manages user interaction, stores session state, and routes tasks to the correct agents or tools.

The second layer is the core workflow layer, including workflow.py and campaign workflow logic. This layer organizes the sequence of search, ranking, CV preparation, drafting, and tracking.

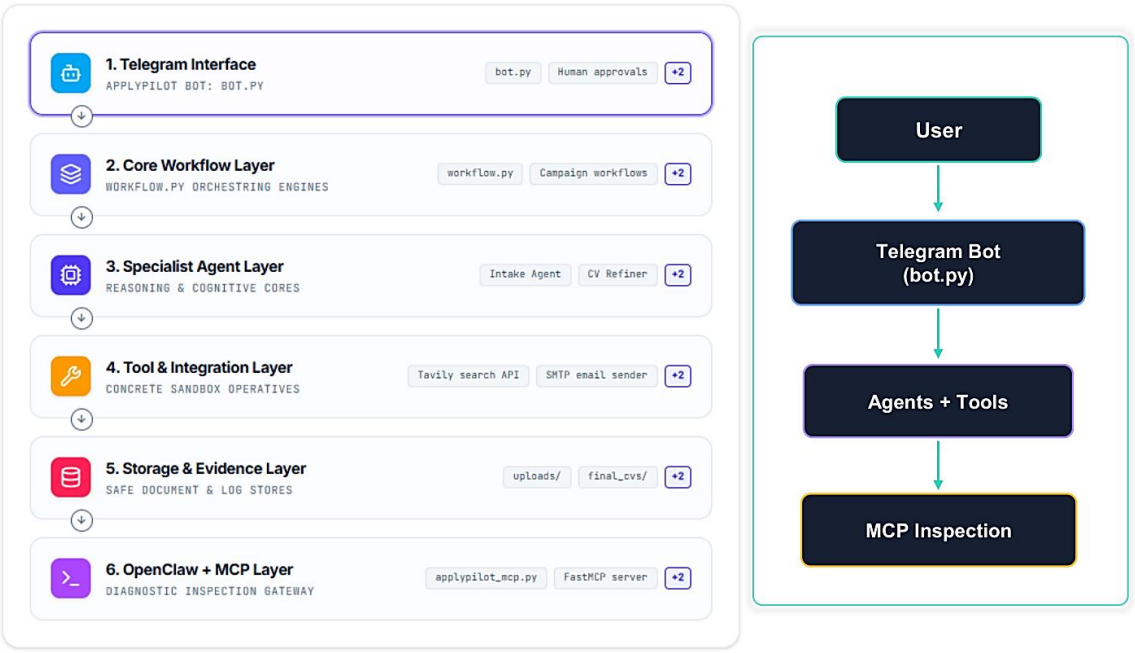


Fig. 1. ApplyPilot system architecture. The Telegram bot acts as the main interface and orchestrator, while agents and tools execute specialized tasks. The MCP layer provides inspection and diagnostic access.

The specialist agent layer contains the modules responsible for reasoning and transformation. The intake and analysis agents support user-goal understanding and opportunity ranking. The CV refiner agent produces tailored CV drafts and revisions. The draft agent generates cover letters and email bodies. The Evaluation Agent checks whether the final CV, cover letter, email body, and subject are ready for sending. The chat agent supports normal conversation when the user is not issuing workflow commands.

The tool and integration layer connect ApplyPilot to external or local capabilities. Tavily is used for live opportunity search. Ollama is used as the local worker model for private CV refinement and draft generation to ensure the privacy of the user's personal information in the CV. The CV reader extracts text from uploaded CV files. The document exporter creates DOCX and PDF outputs. Gmail SMTP sends approved application emails, while Gmail IMAP checks replies. The CSV tracker stores application IDs, recipient status, sent time, reply status, reply subject, reply preview, and notification status.

Fig. 2 summarizes the tools, models, and evidence paths used in the implementation.

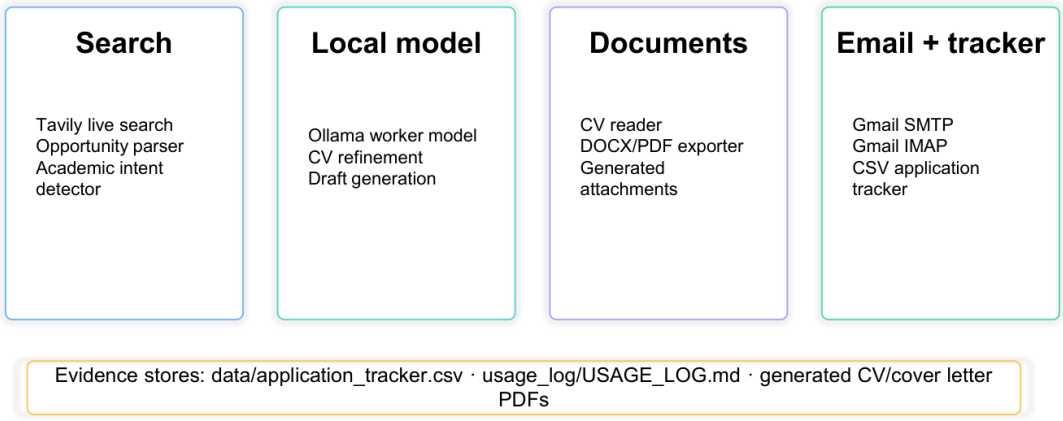


Fig. 2. Tools, models, and evidence paths used in ApplyPilot.

The storage and evidence layer contains generated CVs, cover letters, uploaded files, usage logs, session logs, and application_tracker.csv. This evidence layer is important because the project is evaluated not only by whether the bot responds, but by whether it creates auditable records of real workflow use.

ApplyPilot also includes an OpenClaw as an MCP inspection layer. This layer is implemented through a custom FastMCP server, mcp_server/applypilot_mcp.py. The MCP layer does not replace the main Telegram workflow. Instead, it exposes safe inspection tools, such as status checks, latest logs, and tracker summaries as shown in Fig. 3. In this design, Telegram remains the primary action interface, while MCP provides a standardized inspection and diagnostic gateway for project artifacts.

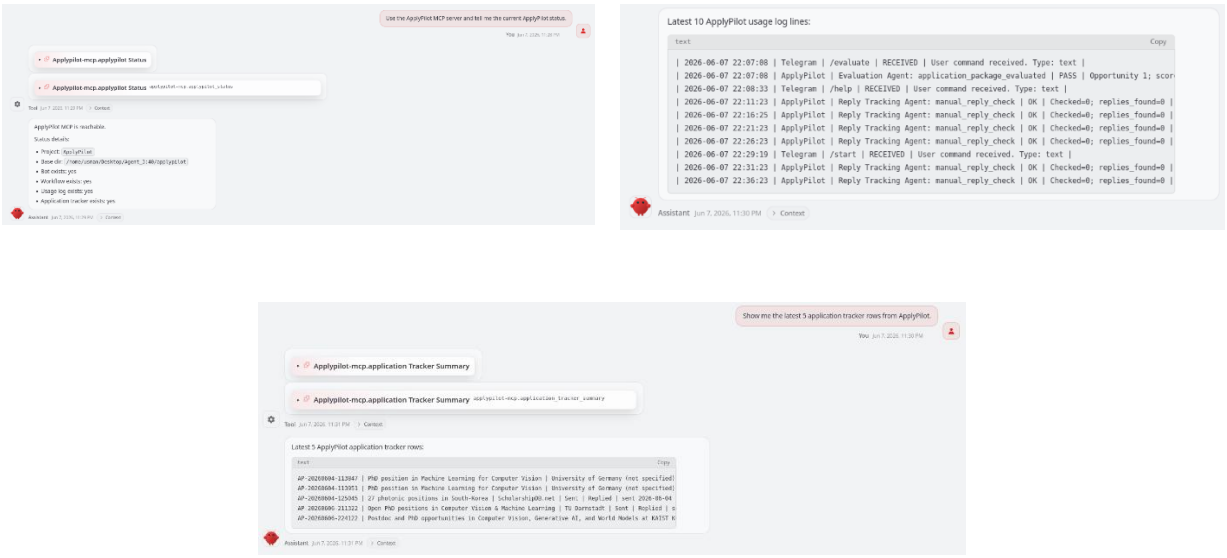


Fig. 5. Working of the OpenClaw to diagnose/inspect the status of the ApplyPilot.

The project structure reflects this architecture. The agents/ folder contains specialist agents; tools/ contains integrations and utilities; agent_cards/ and skills/ document agent roles and reusable workflows; docs/ contains architecture, data flow, cost, threat model, and workflow documents; and usage_log/USAGE_LOG.md provides daily-use evidence.

IV. Evaluation and Results

The system was evaluated through functional end-to-end testing using a real Telegram workflow. The test demonstrated that ApplyPilot can complete the major stages required for application automation.

Fig. 4 provides a condensed end-to-end workflow summary reconstructed from the live test sequence. The workflow starts when the user begins campaign mode and uploads a CV. An academic goal such as "Computer vision PhD Germany" is detected correctly, and the search pipeline retrieves and ranks six candidate opportunities. The system then produces a tailored CV draft, supports revision through user feedback, finalizes the selected CV, generates the cover letter and email, evaluates the application package, sends the email with PDF attachments, and finally tracks inbox replies.

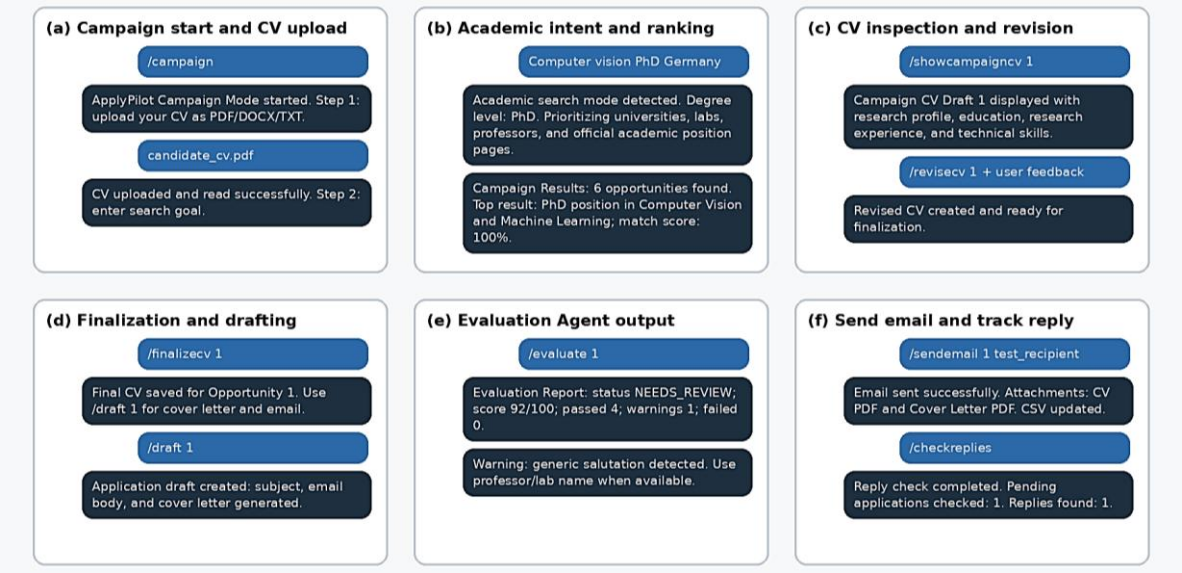


Fig. 4. End-to-end ApplyPilot workflow demonstration reconstructed from the live test sequence.

In addition to the reconstructed workflow summary, Fig. 5 shows evidence of the actual Telegram workflow from the live ApplyPilot execution. It captures the start of campaign mode, CV upload, academic intent detection, ranked campaign results, CV inspection, revision, finalization, draft generation, Evaluation Agent output, email sending, and reply tracking. Together, Figs. 3 and 4 provide both an abstract and a concrete view of the deployed workflow.

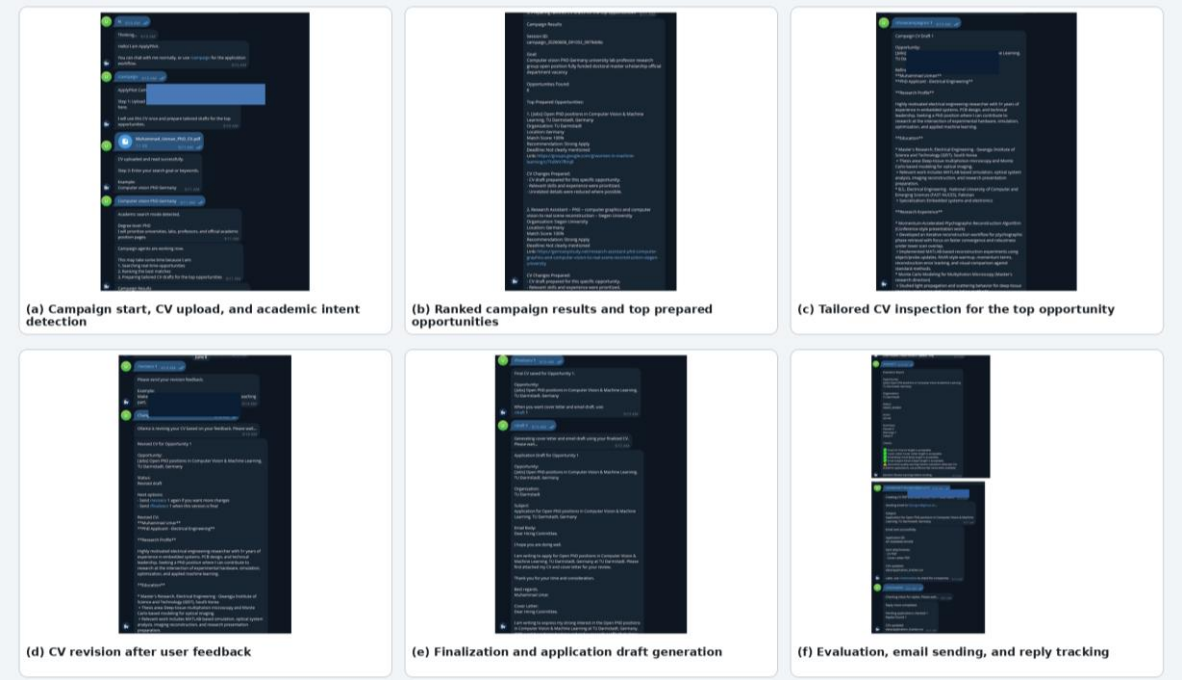


Fig. 5. Actual Telegram workflow evidence showing campaign start, ranked opportunity retrieval, CV inspection and revision, application draft generation, evaluation, email sending, and reply tracking.

Table I summarizes the main functional evaluation.

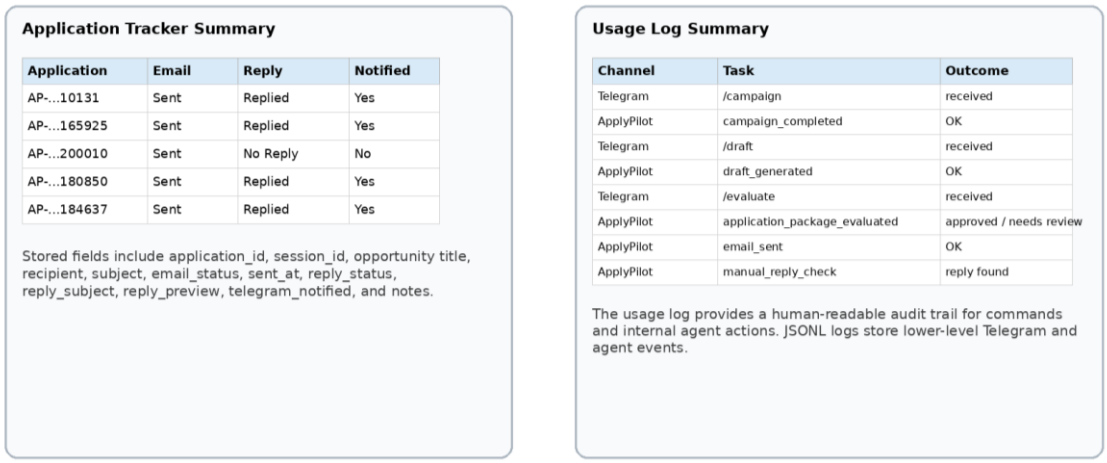
Table I. Functional evaluation summary.

Function	**Test evidence**	**Result**
Normal chat	Telegram greeting and response	Passed
Campaign mode	/campaign started and requested CV	Passed
CV upload	PDF CV uploaded and read	Passed
Academic intent detection	“Computer vision PhD Germany” detected as PhD academic query	Passed
Opportunity search/ranking	Six opportunities found and ranked	Passed
CV tailoring	Tailored CV draft generated	Passed
CV revision	User feedback applied	Passed
CV finalization	Final CV saved	Passed
Draft generation	Email and cover letter generated	Passed
Evaluation Agent	Score 92/100, 0 failed checks	Passed with warning
Email sending	CV and cover letter PDFs sent	Passed
Reply tracking	One reply detected and tracker updated	Passed
Evidence logging	Usage log and CSV tracker populated	Passed

The CSV tracker provided additional evidence of system operation. It stored application IDs, session IDs, opportunity numbers, opportunity titles, recipient fields, email status, sent timestamps, reply status, reply subject, reply preview, Telegram notification status, notification timestamps, and notes. The tracker showed multiple test applications with sent status, several replied statuses, and recorded reply previews. This confirms that ApplyPilot does not only send emails, but also maintains application history and reply evidence.

The usage log also confirmed repeated interaction with the system. Logged entries included /campaign, /showcampaigncv, /revisecv, /finalizecv, /draft, /evaluate, /sendemail, /checkreplies, /status, manual reply checks, search actions, campaign completion, draft generation, and evaluation events. These records support the claim that ApplyPilot provides an auditable workflow rather than a one-time demonstration.

These evidence records are illustrated in Fig. 6, which consolidates excerpts from the application tracker and usage log. The Fig. shows that sent status, reply status, reply previews, and Telegram notification fields are captured alongside agent-level activity records.



Application Tracker Summary

Application	Email	Reply	Notified
AP-...10131	Sent	Replied	Yes
AP-...165925	Sent	Replied	Yes
AP-...200010	Sent	No Reply	No
AP-...180850	Sent	Replied	Yes
AP-...184637	Sent	Replied	Yes

Stored fields include application_id, session_id, opportunity title, recipient, subject, email_status, sent_at, reply_status, reply_subject, reply_preview, telegram_notified, and notes.

Usage Log Summary

Channel	Task	Outcome
Telegram	/campaign	received
ApplyPilot	campaign_completed	OK
Telegram	/draft	received
ApplyPilot	draft_generated	OK
Telegram	/evaluate	received
ApplyPilot	application_package_evaluated	approved / needs review
ApplyPilot	email_sent	OK
ApplyPilot	manual_reply_check	reply found

The usage log provides a human-readable audit trail for commands and internal agent actions. JSONL logs store lower-level Telegram and agent events.

Fig. 6. Application tracker and usage log evidence supporting auditability and reply-tracking claims.

Figs. 7 and 8 add the actual evidence files used during testing. Fig. 6 presents application_tracker.csv screenshots, showing persistent application IDs, session identifiers, opportunity titles, sent states, reply states, generated file paths, and Telegram notification fields. Fig. 7 presents usage-log evidence, showing repeated workflow commands and agent-level actions such as campaign execution, draft generation, evaluation, email sending, and reply checking.

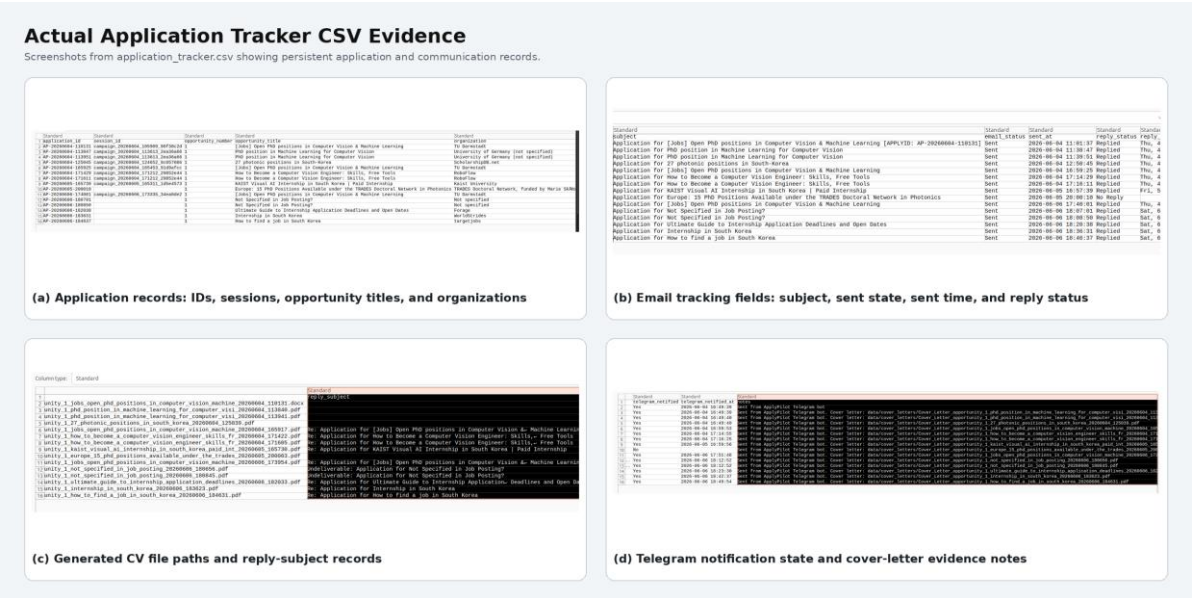


Fig. 7. Actual application tracker evidence showing stored opportunity records, send/reply states, generated files, and notification fields.

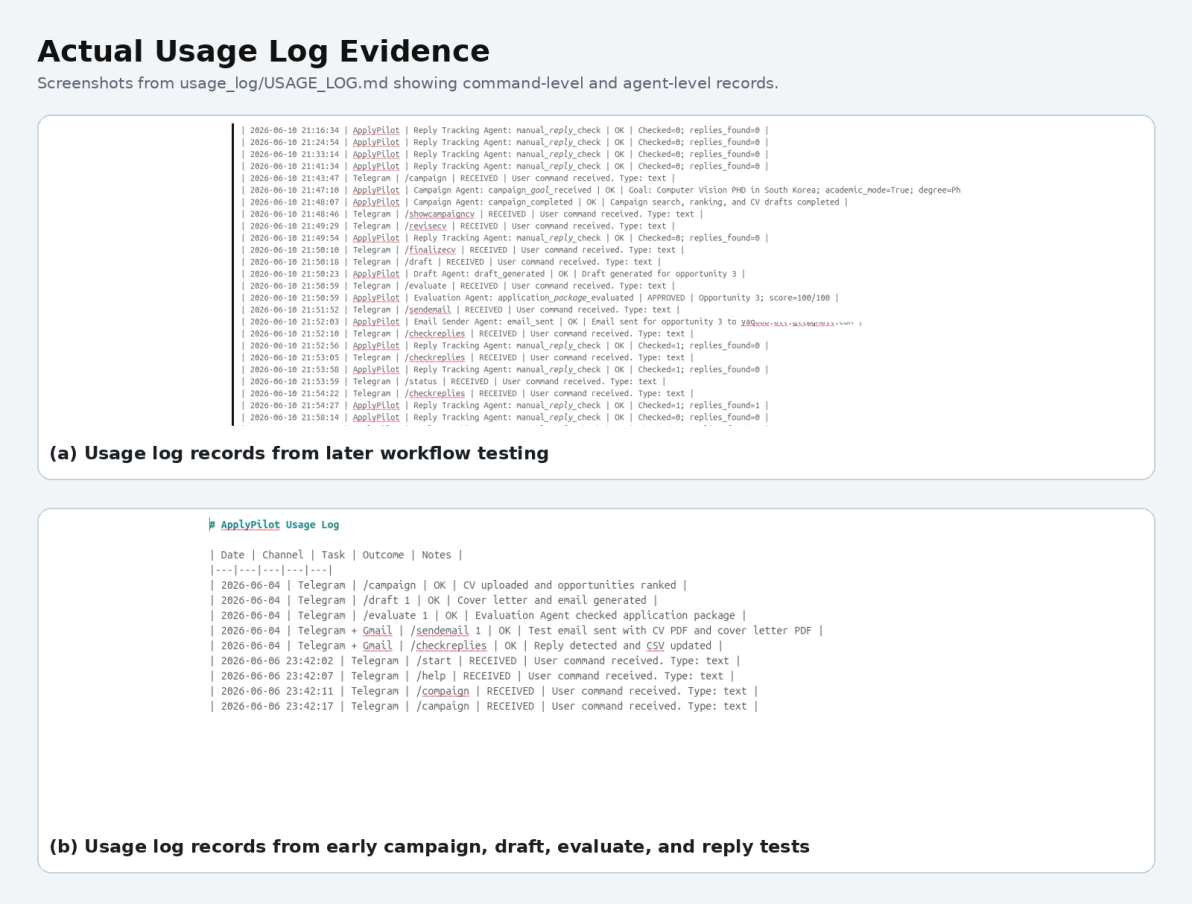


Fig. 8. Actual usage-log evidence showing campaign, drafting, evaluation, email sending, and reply-tracking records.

V. Discussion and Limitations

ApplyPilot demonstrates that a practical private AI assistant can automate a multi-step application workflow while preserving user control. The most useful design choice was separating the workflow into agents and tools rather than relying on a single chat response. This made it possible to add approval gates, evaluation checks, logging, and reply tracking.

The use of a local Ollama model for CV refinement and drafting supports privacy because sensitive CV content can be processed locally. External tools are used only where needed: Tavily for live search and Gmail for email operations. This separation supports a two-tier architecture where private generation and external integrations are handled differently.

The Evaluation Agent is also important. Without evaluation, the assistant could generate and send weak or incomplete application materials. By checking CV length, cover letter length, email length, subject quality, unwanted phrases, and salutation quality, the Evaluation Agent adds an explicit verification stage before email sending. Although the current checks are rule-based, they provide a practical safety layer and can be extended with more advanced quality metrics.

However, the current system also has some limitations. Most of these limitations are not caused by the ApplyPilot workflow design itself, but by the use of free or lightweight APIs during implementation. For example, live opportunity search can sometimes return noisy or partially relevant results because the current free search pipeline has limited ranking, filtering, and page-understanding capability. Similarly, academic search results can be improved further by using a stronger paid search or retrieval API that can better identify universities, laboratories, professors, and official vacancy pages. The same architecture can support this improvement with minimal changes, because the search component is already modular and can be replaced with a higher-quality API. In the same way, CV refinement, cover letter generation, and evaluation can become more accurate if the local or free model is replaced with a stronger paid language model. Therefore, a major future improvement is to keep the existing ApplyPilot workflow but upgrade the underlying search and model APIs. This would improve opportunity relevance, country filtering, factual consistency, and document quality without requiring a complete redesign of the system. Email delivery also depends on the conFig. d mail provider, so future deployment may use a more reliable transactional email service instead of a basic Gmail SMTP setup. Overall, ApplyPilot provides a working end-to-end framework, and its performance can be significantly improved by upgrading the external APIs while preserving the same agentic workflow.

VI. Conclusion

This paper presented ApplyPilot, a private agentic assistant for application search, CV tailoring, evaluation, email sending, and reply tracking. ApplyPilot integrates a Telegram interface, live search, academic intent detection, local LLM-based drafting, document export, Gmail communication, CSV tracking, usage logs, and an OpenClaw/MCP inspection layer. Functional testing showed that the system can complete an end-to-end application workflow, including CV upload, opportunity search, CV tailoring, revision, finalization, draft generation, evaluation, email sending, reply detection, and record keeping.

The project demonstrates that practical agentic systems require more than a chatbot interface. They require workflow orchestration, tool integration, state management, evidence logs, human approval gates, and evaluation mechanisms. ApplyPilot provides these components in a working application assistant. Future work will focus on stricter country filtering, deeper academic laboratory search, improved factual verification, and expanded MCP inspection tools.

References

[1] M. Usman and M. Umar, “ApplyPilot: A private agentic assistant for applications, CV tailoring, evaluation, email sending, reply tracking, and MCP-based inspection,” project presentation, Department of Electrical Engineering and Computer Science, Gwangju Institute of Science and Technology, Korea, Jun. 2026.

[2] H.-N. Lee, “Building a Useful Private AI Butler,” GIST GenAI and Blockchain course lecture slides, Gwangju Institute of Science and Technology, May 2026.

[3] H.-N. Lee, “MCP for OpenClaw,” GIST GenAI and Blockchain course lecture slides, Gwangju Institute of Science and Technology, May 2026.

[4] ApplyPilot project repository files, including bot.py, workflow.py, agents/, tools/, usage_log/USAGE_LOG.md, and data/application_tracker.csv, 2026.